



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 7

No de Práctica(s): 3

Integrante(s): 323078401

323115296

323082392

323139441

323253916

*No. de lista o
brigada:* 4

Semestre: 2027-I

Fecha de entrega: 11/09/2026

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción	2
2. Marco Teórico	2
3. Desarrollo	3
3.1. Ejercicio 1: Análisis de las respuestas de las LLM	3
3.1.1. Análisis de la respuesta 1	3
3.1.2. Análisis de la respuesta 2	4
3.1.3. Análisis de la respuesta 3	4
3.1.4. Observaciones	5
3.2. Ejercicio 2: Implementación de la aplicación de palíndromos	5
4. Resultados	7
5. Conclusiones	8

1. Introducción

Planteamiento del problema. En esta práctica se plantea, por un lado, comparar cómo varían las respuestas de distintas LLM ante consultas sobre `HashMap`, `TreeSet` y ejemplos comunes de uso; por otro lado, se requiere implementar una aplicación en Java que determine si un número entero positivo de exactamente cinco dígitos es un palíndromo, implementado con un método estático.

Motivación. Comparar las respuestas generadas por diferentes LLM sobre la explicación de las estructuras de datos `HashMap` y `TreeSet` para analizar la completitud y utilidad de sus respuestas frente a diversas instrucciones. Asimismo, para el aprendizaje de la programación orientada a objetos en Java, dominar el control de flujo, la descomposición aritmética de datos sin recurrir a transformaciones de cadena y la condición de utilizar un método estático.

Objetivos.

- Comparar la calidad de las respuestas generadas por diferentes LLM a partir de la variación de los prompts proporcionados.
- Implementar una aplicación en lenguaje Java que determine si un entero de cinco dígitos es un palíndromo.
- Aplicar una validación de entrada mediante ciclos de control para restringir el dominio de datos a enteros positivos de exactamente cinco dígitos.
- Emplear métodos estáticos para separar la verificación de palíndromos del flujo principal de interacción con el usuario.

2. Marco Teórico

Las colecciones en Java proporcionan estructuras para la manipulación de grupos de objetos bajo distintas reglas de acceso y almacenamiento.

`HashMap` es una implementación de la interfaz `Map` basada en una tabla de dispersión (*hash table*). Almacena elementos en pares clave-valor, donde cada clave es única. Para determinar la posición de almacenamiento de un elemento se aplica la función `hashCode()` sobre la clave y la mapea a una casilla (*bucket*) dentro de un arreglo interno. En presencia de colisiones (cuando dos claves generan la misma casilla), Java gestiona los elementos mediante listas enlazadas o mediante árboles balanceados si la cantidad de colisiones en un mismo *bucket* excede un umbral predeterminado [1].

TreeSet es una implementación de la interfaz **Set** respaldada internamente por un árbol binario de búsqueda auto-balanceado (árbol Rojo-Negro). Esta estructura garantiza que no existan elementos duplicados y los mantiene ordenados continuamente, ya sea mediante su orden natural (implementando la interfaz **Comparable**) o a través de un **Comparator** personalizado. Esta estructura permite realizar consultas eficientes por rango y obtener elementos extremos [2].

Un método estático en Java es un miembro de clase declarado con la palabra clave **static**. Un método estático pertenece a la clase en sí y no a un objeto en particular. Por ello, puede ser invocado sin necesidad de instanciar la clase que lo contiene. Se utiliza principalmente para realizar operaciones auxiliares o de cómputo puro que reciben todos sus datos de entrada mediante parámetros explícitos y no dependen del estado interno de una instancia [3].

3. Desarrollo

3.1. Ejercicio 1: Análisis de las respuestas de las LLM

Se analizó el comportamiento de tres modelos de lenguaje a los que se les presentó la tarea de describir el funcionamiento de las estructuras de datos **HashMap** y **TreeSet**, junto con solicitudes sobre 3 ejemplos o casos de uso.

3.1.1. Análisis de la respuesta 1

En la primera interacción, el prompt combinó de forma ambigua la solicitud de las estructuras de datos con un pedido de tres ejemplos de compilación. En el prompt no se incluyeron restricciones explícitas de lenguaje de programación, formato, ni nivel de profundidad. La LLM asumió Java como lenguaje por defecto entregando una tabla comparativa y fragmentos de código correctos pero desvinculados del problema principal.

La LLM dividió su respuesta en tres secciones. Al comienzo describió el almacenamiento mediante pares clave-valor, el objetivo de búsqueda en tiempo promedio $O(1)$, el uso de funciones hash y la gestión de colisiones (encadenamiento y direccionamiento abierto). Después explicó que almacena elementos únicos basados en un árbol Rojo-Negro, su ordenamiento mediante **Comparable** o **Comparator**, su complejidad de $O(\log n)$ y la restricción respecto a valores nulos. Al final incluyó una tabla comparativa resumida y proporcionó tres ejemplos de *errores de compilación* en Java (incompatibilidad de tipos, símbolo no declarado y omisión de retorno).

3.1.2. Análisis de la respuesta 2

En la segunda interacción, el prompt utilizó un esquema estructurado de preguntas explícitas (qué es, cómo funciona, para qué sirve, cuándo usarlo y ejemplos de aplicación). En el prompt no se especificó un lenguaje único, aunque las preguntas detonantes acotaron el rango de respuesta para evitar divagaciones.

La LLM entregó una explicación profunda que detalló aspectos internos como el factor de carga, la conversión de *buckets* a árboles rojo-negro, métodos de navegación por rango y reglas claras de decisión para la selección de estructuras en proyectos reales.

La LLM generó una respuesta dividida sistemáticamente según los cuestionamientos del prompt. Al comienzo definió la interfaz Map, detalló su funcionamiento interno: array de buckets, `hashCode()`, factor de carga/*rehashing*, resolución de colisiones y conversión de listas a árboles rojo-negro en Java 8+. Su complejidad $O(1)$ promedio, $O(n)$ peor caso; y los casos donde debe usarse. Después definió la interfaz Set, su respaldo interno en un TreeMap (árbol rojo-negro), el ordenamiento mediante Comparable/Comparator, su complejidad constante de $O(\log n)$ y sus métodos de consulta por rango (`higher`, `lower`, `headSet`, etc.). Al final presentó una tabla comparativa y detalló 3 ejemplos reales de aplicación para HashMap: caché, conteo de frecuencias, diccionarios; y 3 para TreeSet: leaderboards, calendarios/agendas, eliminación de duplicados ordenados. Finalizó con reglas prácticas de decisión.

3.1.3. Análisis de la respuesta 3

En la tercera interacción, el prompt definió un rol claro solicitando la explicación desde la perspectiva de un estudiante de ingeniería. La LLM adaptó la respuesta utilizando un enfoque pedagógico que incluyó fragmentos de código funcional y esquemas conceptuales en texto plano. La frase “cuéntame sobre ellos” es abierta, pero la restricción del perfil “como estudiante de ingeniería” acota la perspectiva hacia los fundamentos conceptuales y algorítmicos.

La LLM adaptó su tono a uno académico-pedagógico. Al comienzo situó ambas estructuras dentro del Java Collections Framework. Explicó HashMap mediante sintaxis de código en Java (`<K,V>`, `.put()`, `.get()`) y esquemas visuales simples. Continuó con TreeSet, mostrando el filtrado de duplicados y ordenamiento mediante un diagrama conceptual de árbol binario. Después utilizó diagramas ASCII para contrastar las estructuras y desarrolló 3 ejemplos prácticos: dos para HashMap: asociación de ID-objeto y conteo de frecuencias; y uno para TreeSet, ordenamiento automático de puntuaciones. Concluyó integrando el tema con la asignatura de Estructuras de Datos y Algoritmos (EDA), enfatizando la diferencia de eficiencias $O(1)$ vs. $O(\log n)$.

3.1.4. Observaciones

El Prompt 1 mezcló dos peticiones inconexas y usó una frase ambigua (“ejemplos de compilación”). El Prompt 2 utilizó un esquema de preguntas directas. El Prompt 3 adoptó un estilo conversacional definiendo el rol del usuario (“estudiante de ingeniería”). Aparte, el Prompt 2 fue el más exigente al solicitar explícitamente el “qué, cómo, para qué, por qué, dónde y cuándo”. Los Prompts 1 y 3 fueron más abiertos.

La Respuesta 1 ofreció un marco teórico estándar acompañado de errores de compilación no relacionados con las estructuras. La Respuesta 2 ofreció el nivel técnico más detallado, abordando conceptos de bajo nivel como *treeification*, *load factor* y rebalanceo de árboles. La Respuesta 3 se enfocó en la parte pedagógica con ejemplos de código y diagramas. Por lo que, la respuesta 2 resultó ser la más adecuada para obtener un análisis técnico exhaustivo, preciso y aplicable de las estructuras de datos HashMap y TreeSet. Su fortaleza radicó en las preguntas guía explícitas que forzaron a la LLM a cubrir desde los mecanismos internos de memoria hasta los criterios de selección en el desarrollo de software. Por su parte, el resultado 3 ofreció el mejor enfoque desde una perspectiva pedagógica para el nivel universitario de tercer semestre, al conectar el uso de las estructuras con la materia de Estructuras de Datos y Algoritmos y proporcionar ejemplos de código claros mientras que el resultado 1 fue el menos satisfactorio debido a la ambigüedad en la formulación de sus instrucciones, lo que provocó una respuesta fragmentada en dos temas sin conexión evidente.

3.2. Ejercicio 2: Implementación de la aplicación de palíndromos

Para dar solución al problema planteado por el profesor, se desarrolló una aplicación en Java estructurada en una clase principal.

El programa inicia solicitando al usuario el ingreso de un número entero por consola mediante la clase `Scanner`. Inmediatamente después de la captura inicial, se ejecuta una estructura `if` para eliminar signos negativos, continúa con una estructura `while` que evalúa si el valor está fuera del rango permitido, menor a 10000 o mayor a 99999. Si la entrada es inválida, el bucle despliega un mensaje explícito de error y vuelve a solicitar el dato, repitiendo esta acción hasta que se introduzca un entero de exactamente cinco dígitos positivos.

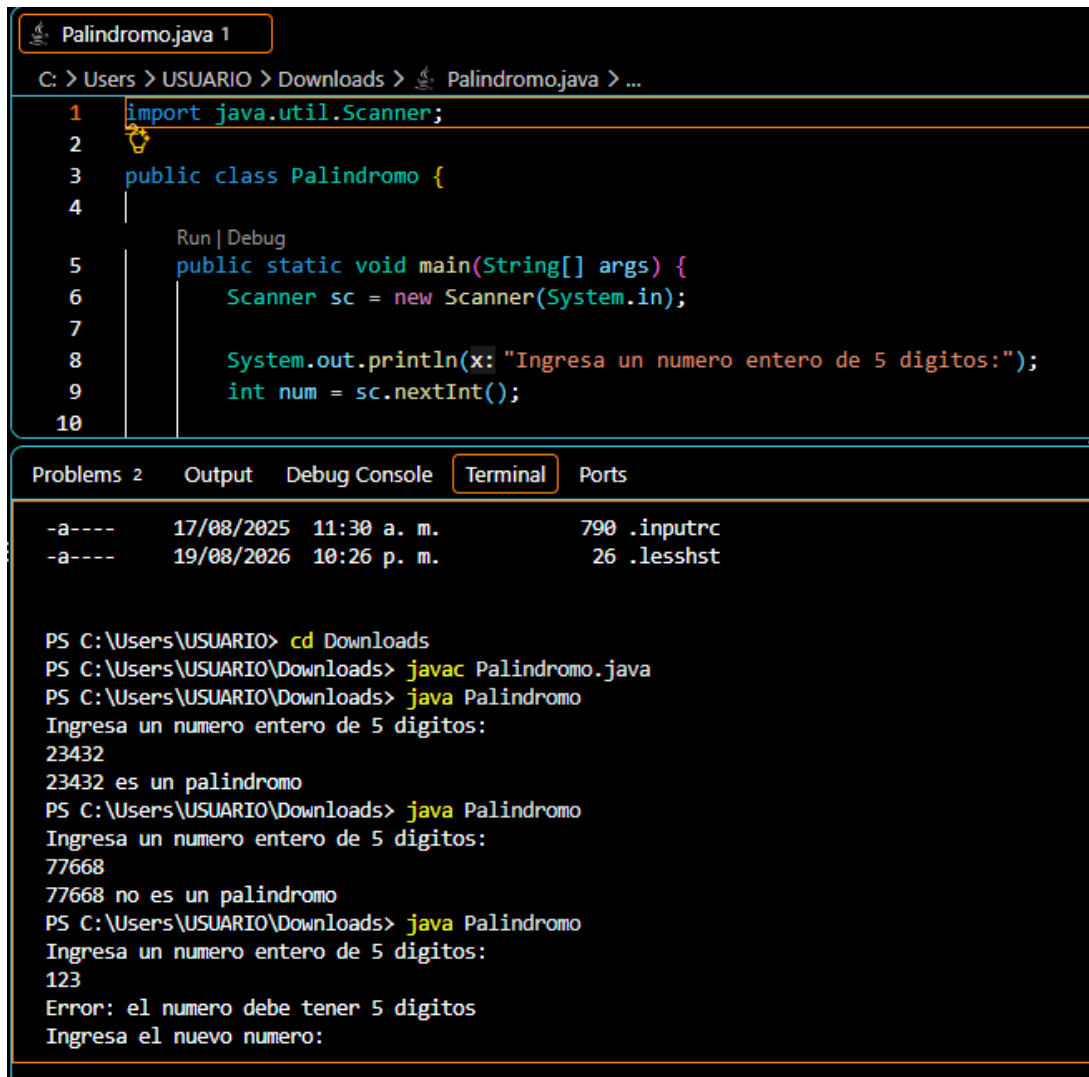
Una vez obtenido un valor válido, la variable se envía como argumento a un método estático de evaluación booleana. Dentro de este método no se utilizan cadenas de texto ni arreglos; el procesamiento se realiza exclusivamente mediante expresiones aritméticas: se extrae el primer dígito mediante la división entera del número entre 10000 y el quinto dígito evaluando el residuo del número módulo 10 para comparar si el primer y el quinto dígito

son iguales. Luego, extrae el segundo dígito dividiendo el número entre 1000 y aplicando el módulo 10 al resultado, después se extrae el cuarto dígito dividiendo el número entre 10 y aplicando módulo 10 al resultado. Esto para comparar si el segundo y el cuarto dígito son iguales.

El método evalúa la conjunción lógica (operador `&&`) de ambas comparaciones. Si ambas parejas de dígitos coinciden, el método retorna un valor verdadero; de lo contrario, retorna falso. El método principal recibe este resultado booleano y, mediante una estructura condicional `if-else`, imprime en consola el mensaje final indicando si el entero ingresado es o no un palíndromo.

Se realizaron pruebas de ejecución introduciendo enteros fuera de rango para verificar la activación del ciclo de reintento, así como valores válidos que correspondían a números palíndromos y no palíndromos.

4. Resultados



The screenshot shows an IDE with a file named `Palindromo.java`. The code is as follows:

```
1 import java.util.Scanner;
2
3 public class Palindromo {
4     Run | Debug
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7
8         System.out.println(x: "Ingresa un numero entero de 5 digitos:");
9         int num = sc.nextInt();
10    }
```

Below the code, the **Terminal** tab is active, showing the following output:

```
-a---- 17/08/2025 11:30 a. m. 790 .inputrc
-a---- 19/08/2026 10:26 p. m. 26 .lessht

PS C:\Users\USUARIO> cd Downloads
PS C:\Users\USUARIO\Downloads> javac Palindromo.java
PS C:\Users\USUARIO\Downloads> java Palindromo
Ingresa un numero entero de 5 digitos:
23432
23432 es un palindromo
PS C:\Users\USUARIO\Downloads> java Palindromo
Ingresa un numero entero de 5 digitos:
77668
77668 no es un palindromo
PS C:\Users\USUARIO\Downloads> java Palindromo
Ingresa un numero entero de 5 digitos:
123
Error: el numero debe tener 5 digitos
Ingresa el nuevo numero:
```

Figura 1: Muestra de entrada los datos que ingresa son digitos menores a 5 o mayores te pide otra vez que ingreses un palindromo de 5 digitos.

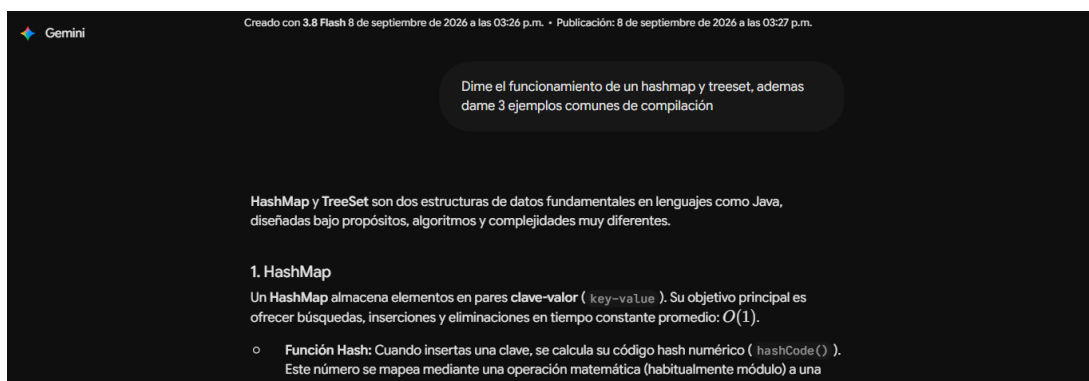


Figura 2: Respuesta de Gemine al Prompt 1: Explica las definiciones para hash mapa y solo da ejemplos teoricos nadamas..

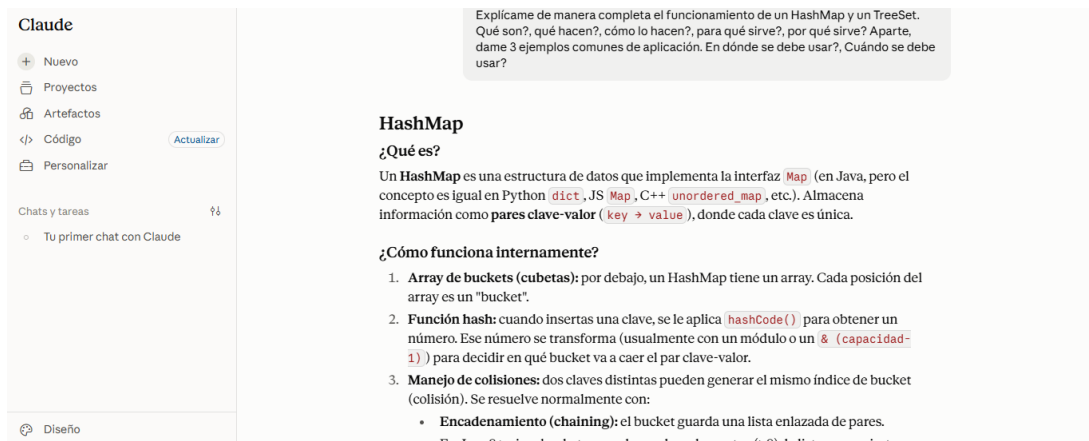


Figura 3: Respuesta de Claude al Prompt 2: Te da un pseudocodigo de hash map usos y sus definiciones

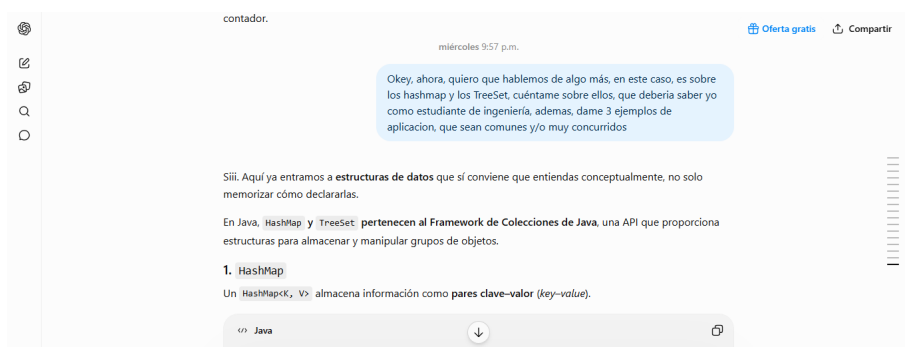


Figura 4: Respuesta de Chat GPT al Prompt 3:Nos regresa codigos en los que se explican el uso del Hash map

En el siguiente enlace se encuentra disponible el diagrama de flujo correspondiente al ejercicio 2: [Ver diagrama en el repositorio de GitHub](#).

Explica graficamente como funciona el programa, los datos de entrada y el mensaje de salida así como sus excepciones de 5 dígitos

5. Conclusiones

Los resultados obtenidos en las ejecuciones de la aplicación confirman que la descomposición aritmética directa permite evaluar la simetría de un número de cinco dígitos de forma precisa y sin requerir estructuras de datos auxiliares. Aparte, la integración de la estructura repetitiva `while` resultó determinante para garantizar la validez de las entradas, impidiendo que datos de longitud incorrecta ingresaran a la fase de procesamiento matemático. Asimismo, la separación del programa en un método estático permitió que la función de evaluación sea independiente de la entrada y salida gestionada en el método principal.

Respecto al análisis de los modelos de lenguaje, la comparación entre las respuestas evidenció que la profundidad técnica de la información obtenida depende directamente de la especificidad de lo que se solicita. Los prompts que incluyeron preguntas guía o roles definidos produjeron explicaciones técnicamente más profundas sobre el comportamiento interno de HashMap y TreeSet, en contraste con aquellos que contenían instrucciones ambiguas.

Referencias

- [1] Oracle, “HashMap (Java Platform SE 8),” Oracle System Documentation, 2024. [En línea]. Disponible: <https://docs.oracle.com/javase/8/docs/api/java/util/HashMap.html>.
- [2] Oracle, “TreeSet (Java Platform SE 8),” Oracle System Documentation, 2024. [En línea]. Disponible: <https://docs.oracle.com/javase/8/docs/api/java/util/TreeSet.html>.
- [3] Oracle, “Understanding Class Members,” The Java™ Tutorials, 2024. [En línea]. Disponible: <https://docs.oracle.com/javase/tutorial/java/java00/classvars.html>.